



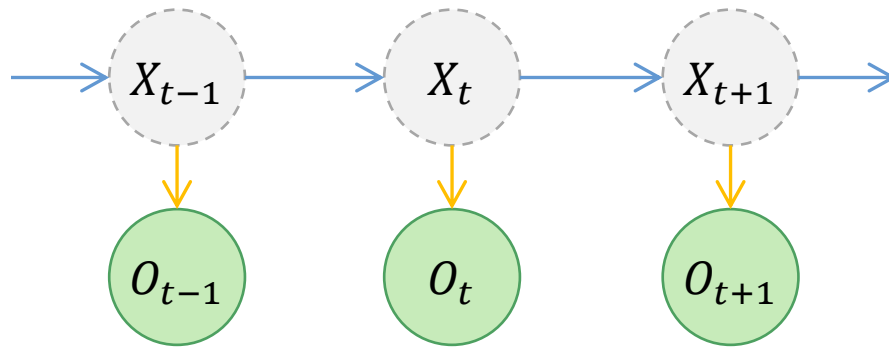
Hidden Markov Models (HMM)

Tutorial

DD2380 Artificial Intelligence
Rebecka Winqvist, rebwin@kth.se

Hidden Markov Models

A Hidden Markov Model (HMM) is a statistical model that is used to describe dynamical systems where the underlying states are not directly observable but can only be inferred from their measurable effects or outputs.



Markov property (*first order*)

The current state X_t depends only on the previous state X_{t-1} (and not on earlier states).

...in other words

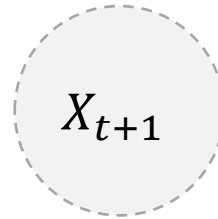
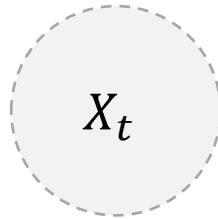
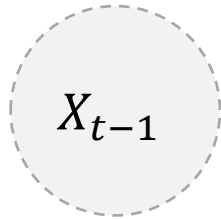
The future is **conditionally independent** of the past **given** the current state:

$$P(X_t \mid X_{t-1}, X_{t-2}, X_{t-3}, \dots) = P(X_t \mid X_{t-1})$$

Elements of an HMM

States

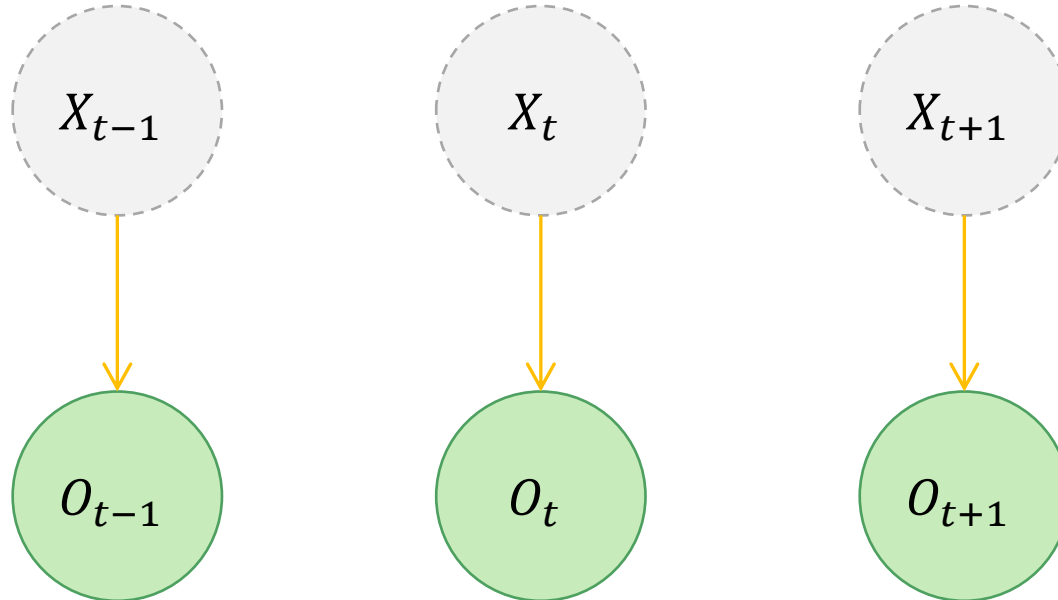
- $\mathcal{X} = \{x_1, x_2, \dots, x_N\}$: set of N states representing the unobservable components of the system
- $X_t \in \mathcal{X}$: the state at time t
- $\pi = [\pi_1 \quad \pi_2 \quad \dots \quad \pi_N]$: initial state distribution s.t. $\pi_i = P(X_1 = x_i)$ and $\sum \pi_i = 1$



Elements of an HMM

Emissions (*observations, outputs*)

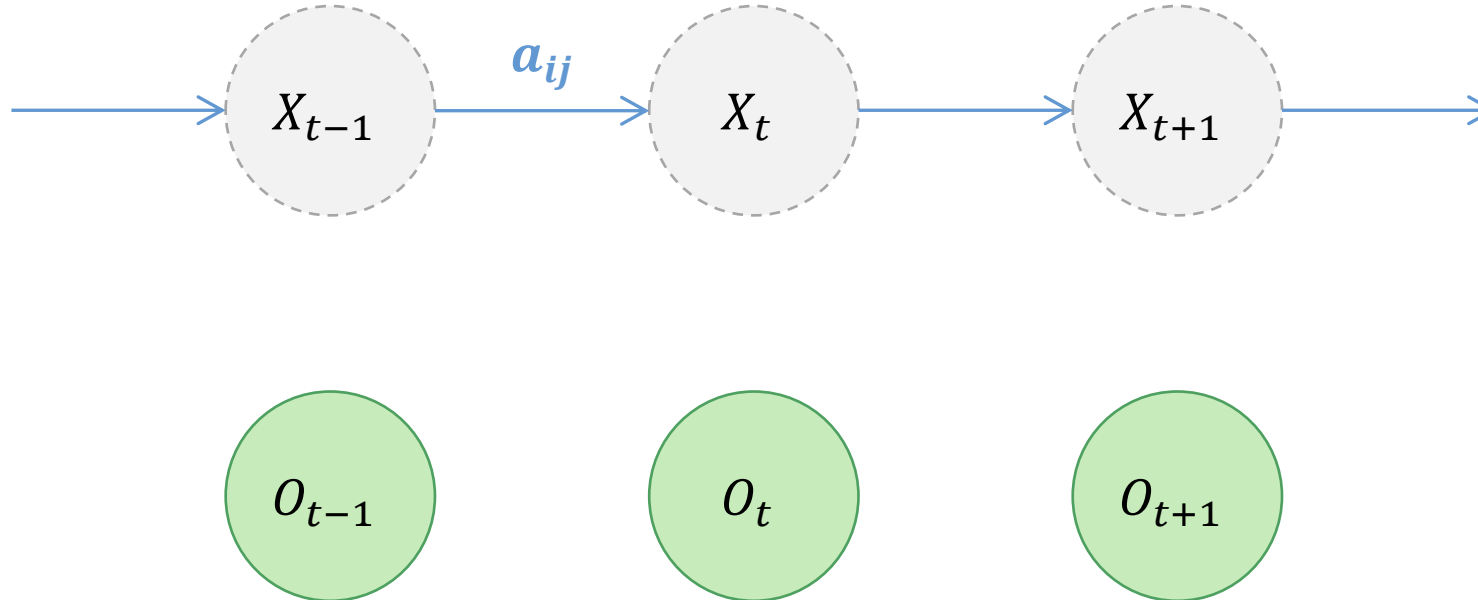
- $\mathcal{O} = \{o_1, o_2, \dots, o_M\}$: set of M observations representing the measurable outcomes generated by the system
- $O_t \in \mathcal{O}$: the observation at time t
- O_t independent of all else given X_t



Elements of an HMM

State transition probabilities

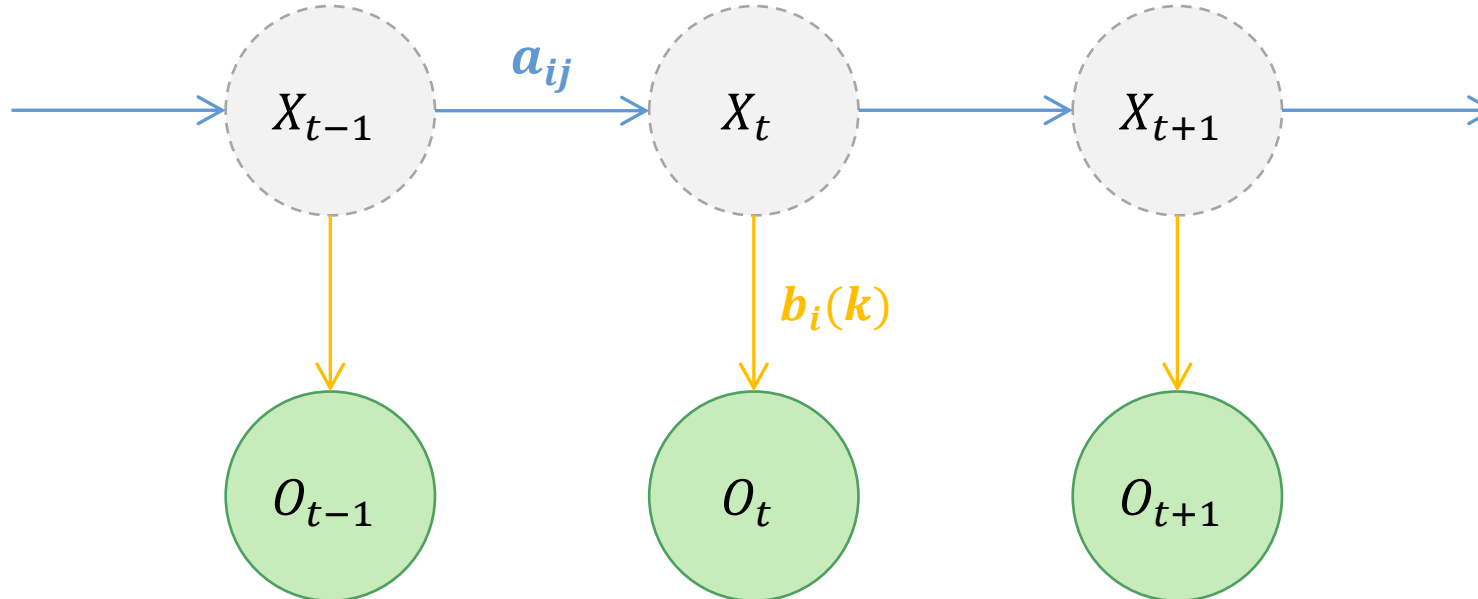
- $a_{ij} = P(X_{t+1} = x_j \mid X_t = x_i) \geq 0$: the likelihood of transitioning from a hidden state i to a hidden state j
- $A = \{a_{ij}\}$, $i, j \in [1, N]$: the transition matrix
- A is row stochastic: $\sum_{j=1}^N a_{ij} = 1$, $\forall i$



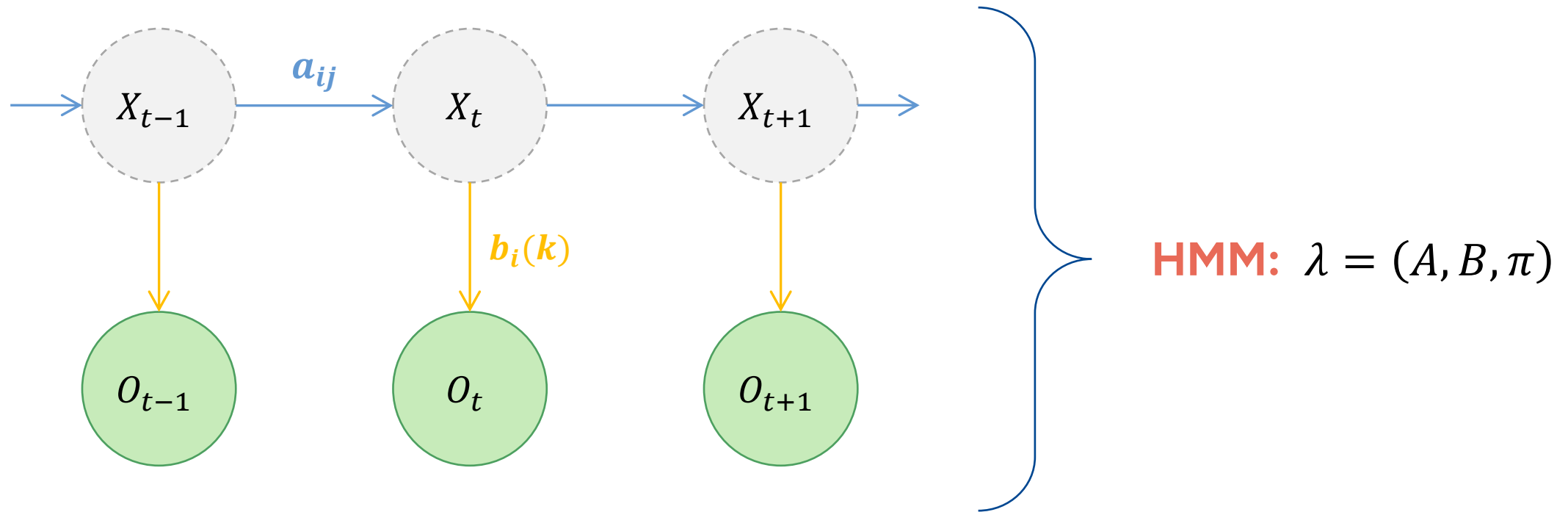
Elements of an HMM

Emission probabilities

- $b_i(k) = P(O_t = o_k \mid X_t = x_i) \geq 0$: the likelihood of observing outcome k in state i
- $B = \{b_i(k)\}$, $i \in [1, N]$, $k \in [1, M]$: the emission matrix
- B is row stochastic: $\sum_{k=1}^M b_i(k) = 1$



HMM representation



Example: Pepper the Pug

Pepper is a two-month-old Pug who lives with her owner, Alex.

Pepper spends her days engaging in typical dog activities. When she's feeling happy, she usually plays, and when she is bored, she typically explores her environment to find something to do.

If Pepper gets hungry, she lets Alex know by barking. After a few hours of playing, Pepper usually becomes tired and takes a nap to rest and recharge.

To better understand Pepper's behavior and anticipate her needs, Alex decides to **model Pepper's daily routine using a Hidden Markov Model**.



Example: Pepper the Pug

Identifying the HMM components

Pepper spends her days engaging in typical dog activities. When she's feeling excited, she usually plays, and when she is bored, she typically explores her environment to find something to do.

If Pepper gets hungry, she lets Alex know by barking. After a few hours of playing, Pepper usually becomes tired and takes a nap to rest and recharge.

States (what is hidden)

Pepper's mood

Example: Pepper the Pug

Identifying the HMM components

*Pepper spends her days engaging in typical dog activities. When she's feeling **excited**, she usually plays, and when she is **bored**, she typically explores her environment to find something to do.*

*If Pepper gets **hungry**, she lets Alex know by barking. After a few hours of playing, Pepper usually becomes **tired** and takes a nap to rest and recharge.*

States (what is hidden)

Pepper's mood

[**E**xcited, **B**ored, **H**ungry, **T**ired]

Example: Pepper the Pug

Identifying the HMM components

*Pepper spends her days engaging in typical dog activities. When she's feeling **excited**, she usually plays, and when she is **bored**, she typically explores her environment to find something to do.*

*If Pepper gets **hungry**, she lets Alex know by barking. After a few hours of playing, Pepper usually becomes **tired** and takes a nap to rest and recharge.*

States (what is hidden)

Pepper's mood

[**E**xcited, **B**ored, **H**ungry, **T**ired]

Emissions (what we observe)

Pepper's behavior

Example: Pepper the Pug

Identifying the HMM components

*Pepper spends her days engaging in typical dog activities. When she's feeling **active**, she usually **plays**, and when she is **bored**, she typically **explores** her environment to find something to do.*

*If Pepper gets **hungry**, she lets Alex know by **barking**. After a few hours of playing, Pepper usually becomes **tired** and takes a **nap** to rest and recharge.*

States (what is hidden)

Pepper's mood

[**A**ctive, **B**ored, **H**ungry, **T**ired]

Num states: $N = 4$

A : $N \times N = 4 \times 4$ matrix

Emissions (what we observe)

Pepper's behavior

[**P**lay, **E**xplore, **B**ark, **N**ap]

Num emissions: $M = 4$

B : $N \times M = 4 \times 4$ matrix

HMM Problems

Three (four) problems of interest...

0. Prediction:

- predicting most likely current emission
- predicting most likely next state

1. **Evaluation:** how likely is an emission sequence for an HMM?

2. **Decoding:** what state sequence most likely produced the emission sequence?

3. **Learning:** what model parameters best describe how the emission sequence came about?

Let's take a look at these problems in the Pepper the Puppy example!

Prediction

- Predict most likely emission O_t
- Predict most likely next state X_{t+1} *(see lecture notes)*

Given

- HMM parameters A, B
- Distribution of states at time t : $P(X_t)$

Find

- Most likely next emission at time t : $P(O_t \mid A, B, P(X_t))$

Prediction (Pepper the Pug example)

Given

$$A = \begin{matrix} & \text{A} & \text{B} & \text{H} & \text{S} \\ \text{Active} & 0.6 & 0.1 & 0.1 & 0.2 \\ \text{Bored} & 0 & 0.3 & 0.2 & 0.5 \\ \text{Hungry} & 0.8 & 0.1 & 0 & 0.1 \\ \text{Sleepy} & 0.2 & 0 & 0.1 & 0.7 \end{matrix}, \quad B = \begin{matrix} & \text{Play} & \text{Expl.} & \text{Bark} & \text{Nap} \\ \text{A} & 0.6 & 0.2 & 0.1 & 0.1 \\ \text{B} & 0.1 & 0.4 & 0.1 & 0.4 \\ \text{H} & 0 & 0 & 0.7 & 0.3 \\ \text{S} & 0 & 0 & 0.1 & 0.9 \end{matrix}, \quad P(X_t) = [0.4 \quad 0.2 \quad 0.1 \quad 0.3]$$

Find

$$P(O_t \mid A, B, P(X_t))$$

Solution

$$\forall k \in [1, \dots, M]: \quad P(O_t = o_k) = \sum_{i=1}^N P(X_t = x_i) P(O_t = o_k \mid X_t = x_i)$$

See full solution on Canvas.

1. Evaluation

Compute the likelihood of HMM $\lambda = (A, B, \pi)$ producing observation sequence $O_{1:T} = \{O_1, O_2, \dots, O_T\}$

Given

- HMM parameters A, B, π
- Emission sequence $O_{1:T} = \{O_1, O_2, \dots, O_T\}$

Find

- Likelihood $P(O_{1:T} \mid \lambda) = P(O_{1:T} \mid A, B, \pi)$

Solution

Straightforward way: enumerate every possible state sequence $X_{1:T}$ and compute $P(O_{1:T} \mid X_{1:T}, \lambda)$

1. Evaluation

Compute the likelihood of HMM $\lambda = (A, B, \pi)$ producing observation sequence $O_{1:T} = \{O_1, O_2, \dots, O_T\}$

Given

- HMM parameters A, B, π
- Emission sequence $O_{1:T} = \{O_1, O_2, \dots, O_T\}$

Find

- Likelihood $P(O_{1:T} \mid \lambda) = P(O_{1:T} \mid A, B, \pi)$

Solution

Straightforward way: enumerate every possible state sequence $X_{1:T}$ and compute $P(O_{1:T} \mid X_{1:T}, \lambda)$

↳ *Computationally infeasible even for small values of N and T !*

Instead:

- Forward algorithm
- (Backward algorithm)

Forward algorithm (α -pass)

Introduce forward variable

Probability of observing the partial observation sequence $O_{1:t} = \{O_1, O_2, \dots, O_t\}$ **and** being in state i at time t :

$$\alpha_t(i) = P(O_{1:t}, X_t = i \mid \lambda)$$

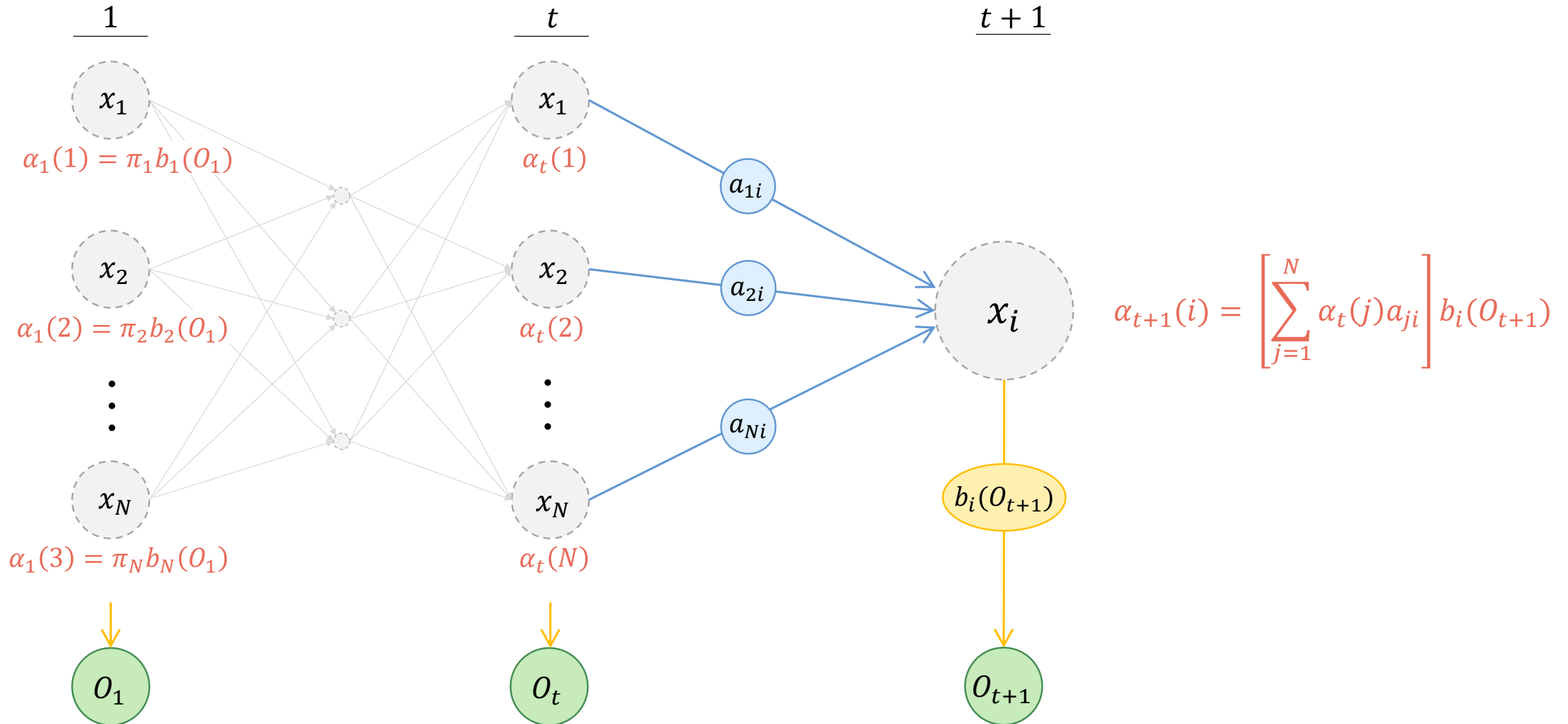
Forward algorithm

- **Initialization:** $\alpha_1(i) = \pi_i b_i(O_1)$

- **Induction:**
$$\alpha_t(i) = \left[\sum_{j=1}^N \alpha_{t-1}(j) a_{ji} \right] b_i(O_t), \quad 2 \leq t \leq T$$

- **Termination:**
$$P(O_{1:T} \mid \lambda) = \sum_{i=1}^N P(O_{1:T}, X_T = i \mid \lambda) = \sum_{i=1}^N \alpha_T(i)$$

Forward algorithm (α -pass)



Backward algorithm (β -pass)

Introduce backward variable

Prob. of generating the remainder of the observation sequence $O_{t+1:T} = \{O_{t+1}, O_{t+2}, \dots, O_T\}$ **given** state i at time t :

$$\beta_t(i) = P(O_{t+1:T} \mid X_t = i, \lambda)$$

Backward algorithm

- **Initialization:** $\beta_T(i) = 1, \forall i \in [1, \dots, N]$

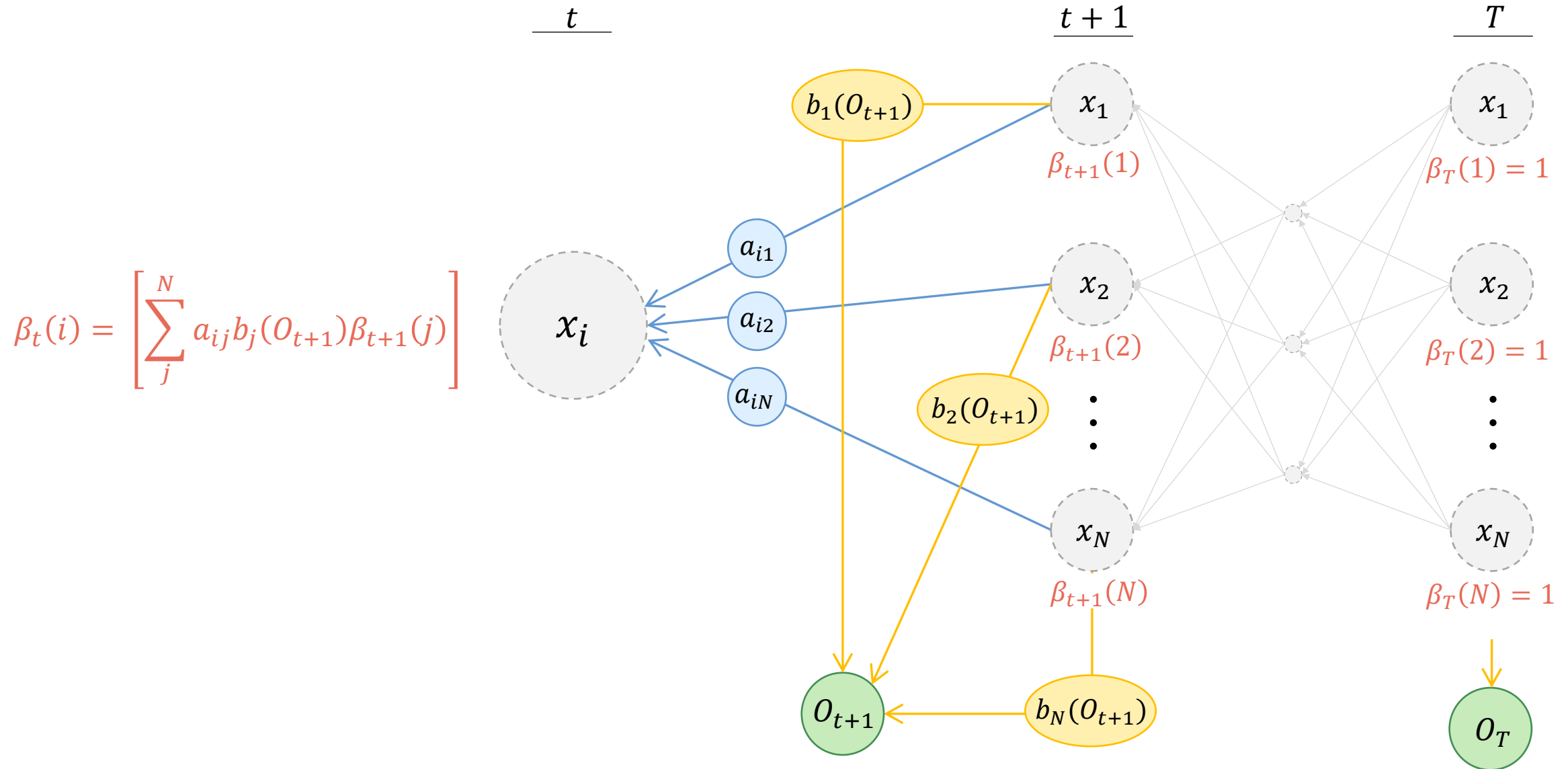
- **Induction:** $\beta_t(i) = \left[\sum_j^N a_{ij} b_j(O_{t+1}) \beta_{t+1}(j) \right], \quad t < T$

- **Termination:** $P(O_{1:T} \mid \lambda) = \sum_{i=1}^N \pi_i b_i(O_1) \beta_1(i)$

NOTE

Backward algorithm is not necessary for solving the Evaluation problem but will be used when solving the Learning problem.

Backward algorithm (α -pass)



Evaluation (Pepper the Pug example)

Given

$$A = \begin{array}{c} \text{Active} \\ \text{Bored} \\ \text{Hungry} \\ \text{Sleepy} \end{array} \begin{array}{c} \text{A} \quad \text{B} \quad \text{H} \quad \text{S} \\ \begin{bmatrix} 0.6 & 0.1 & 0.1 & 0.2 \\ 0 & 0.3 & 0.2 & 0.5 \\ 0.8 & 0.1 & 0 & 0.1 \\ 0.2 & 0 & 0.1 & 0.7 \end{bmatrix} \end{array},$$

$$B = \begin{array}{c} \text{A} \\ \text{B} \\ \text{H} \\ \text{S} \end{array} \begin{array}{c} \text{Play} \quad \text{Expl.} \quad \text{Bark} \quad \text{Nap} \\ \begin{bmatrix} 0.6 & 0.2 & 0.1 & 0.1 \\ 0.1 & 0.4 & 0.1 & 0.4 \\ 0 & 0 & 0.7 & 0.3 \\ 0 & 0 & 0.1 & 0.9 \end{bmatrix} \end{array},$$

$$\pi = \begin{array}{c} \text{A} \quad \text{B} \quad \text{H} \quad \text{S} \\ [0.5 \quad 0 \quad 0 \quad 0.5] \end{array}$$

$$O_{1:T} = \{\text{N}, \text{P}, \text{P}, \text{B}\}$$

Find

$$P(O_{1:T} \mid \lambda) = P(O_{1:T} \mid A, B, \pi)$$

Solution

Forward algorithm. Full solution on Canvas.

2. Decoding

Find the state sequence $X_{1:T}^*$ that is most likely to have produced the observations $O_{1:T}$.

Given

- HMM parameters A, B, π
- Emission sequence $O_{1:T} = \{O_1, O_2, \dots, O_T\}$

Find

- State sequence $X_{1:T}^* = \{X_1, X_2, \dots, X_T\}$ that most likely produced $O_{1:T}$
- Probability of occurrence of $X_{1:T}^*$

Solution

Viterbi algorithm (dynamic programming technique)

Viterbi algorithm

Introduce

- $\delta_t(i) = \max_{X_1, X_2, \dots, X_t} P(X_1, X_2, \dots, X_t = i, O_1, O_2, \dots, O_t \mid \lambda) = \max_{j \in [1, \dots, N]} [\delta_{t-1}(j) a_{ji}] b_i(O_t)$ *(prob. of most likely path that leads to state i at time t)*
- $\psi_t(i)$: back-pointer variable *(keeps track of most likely state sequence)*
: the state at time $t - 1$ with highest prob. of trans. into state i at time t

Viterbi algorithm

- **Initialization:** $\delta_1(i) = \pi_i b_i(O_1)$, $\psi_t(i) = 0$ *(Same as forward algorithm)*
- **Induction:** $\delta_t(i) = \max_{j \in [1, \dots, N]} [\delta_{t-1}(j) a_{ji}] b_i(O_t)$, $\psi_t(i) = \arg \max_j \delta_{t-1}(j) a_{ji}$, $2 \leq t \leq T$
- **Termination:** $P^* = \max_{j \in [1, \dots, N]} \delta_T(j)$, $X_T^* = \arg \max_j \delta_T(j)$
- **Path backtracking:** $X_t^* = \psi_{t+1}(X_{t+1}^*)$

Decoding (Pepper the Pug example)

Given

$$A = \begin{matrix} & \begin{matrix} \text{A} & \text{B} & \text{H} & \text{S} \end{matrix} \\ \begin{matrix} \text{Active} \\ \text{Bored} \\ \text{Hungry} \\ \text{Sleepy} \end{matrix} & \begin{bmatrix} 0.6 & 0.1 & 0.1 & 0.2 \\ 0 & 0.3 & 0.2 & 0.5 \\ 0.8 & 0.1 & 0 & 0.1 \\ 0.2 & 0 & 0.1 & 0.7 \end{bmatrix} \end{matrix},$$

$$B = \begin{matrix} & \begin{matrix} \text{Play} & \text{Expl.} & \text{Bark} & \text{Nap} \end{matrix} \\ \begin{matrix} \text{A} \\ \text{B} \\ \text{H} \\ \text{S} \end{matrix} & \begin{bmatrix} 0.6 & 0.2 & 0.1 & 0.1 \\ 0.1 & 0.4 & 0.1 & 0.4 \\ 0 & 0 & 0.7 & 0.3 \\ 0 & 0 & 0.1 & 0.9 \end{bmatrix} \end{matrix},$$

$$\pi = \begin{matrix} & \begin{matrix} \text{A} & \text{B} & \text{H} & \text{S} \end{matrix} \\ & [0.5 & 0 & 0 & 0.5] \end{matrix}$$

$$O_{1:T} = \{\text{B}, \text{P}, \text{N}, \text{E}\}$$

Find

$$P(X_{1:T}^* \mid O_{1:T}, \lambda) = P(X_{1:T}^* \mid O_{1:T}, A, B, \pi)$$

Solution

Viterbi algorithm. Full solution on Canvas.

3. Learning (or, training)

Find the model parameters $\lambda = (A, B, \pi)$ that maximize the likelihood of observing $O_{1:T} = \{O_1, O_2, \dots, O_T\}$

Given

- Emission sequence $O_{1:T} = \{O_1, O_2, \dots, O_T\}$
- Initial guess of parameters A, B, π

“To Find” (optimize/learn)

Model parameters $\lambda = (A, B, \pi)$ such that $P(O_{1:T} \mid \lambda)$ is maximized

Solution

Baum-Welch algorithm (an Expectation Maximization (EM) technique)

Baum-Welch algorithm

What are we *actually* trying to learn?

- A : transition probabilities
- B : observation probabilities

To this end, we define...

The probability of being in state i at time t and in state j at time $t + 1$, given the model and observation sequence:

Di-gamma function: $\gamma_t(i, j) = P(X_t = x_i, X_{t+1} = x_j \mid O_{1:T}, \lambda) = \frac{\alpha_t(i) a_{ij} b_j(O_{t+1}) \beta_{t+1}(j)}{\sum_{i=1}^N \alpha_T(i)}$

forward variable *backward variable*

The probability of being in state i at time t , given the model and observation sequence:

Gamma function: $\gamma_t(i) = P(X_t = i \mid O_{1:T}, \lambda) = \sum_{j=1}^N \gamma_t(i, j) \quad \left(= \frac{\alpha_t(i) \beta_t(i)}{\sum_{i=1}^N \alpha_t(i) \beta_t(i)} \right)$

Baum-Welch algorithm

Steps:

1. **Initialize:** $\lambda = (A, B, \pi)$
2. **Compute:** $\alpha_t(i), \beta_t(i), \gamma_t(i, j), \gamma_t(i)$
3. **Re-estimate:** $\bar{\lambda} = (\bar{A}, \bar{B}, \bar{\pi})$
4. **Repeat:** from step 2 until $P(O_{1:t} \mid \lambda)$ converges

Re-estimating A, B, π

Recall...

$\gamma_t(i, j)$: prob. of being in state i at time t and state j at time $t + 1$

$\gamma_t(i)$: prob. of being in state i at time t

(given $O_{1:T}$ and λ)

...then,

$\sum_{t=1}^{T-1} \gamma_t(i, j)$: expected number of transitions from state i to state j

$\sum_{t=1}^{T-1} \gamma_t(i)$: expected number of transitions made from state i

Re-estimating A, B, π

Transition probabilities

$$\bar{a}_{ij} = \frac{\text{exp. num of trans. from } i \text{ to } j}{\text{exp. num of trans. from } i} = \frac{\sum_{t=1}^{T-1} \gamma_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}$$

Emission probabilities

$$\bar{b}_i(k) = \frac{\text{exp. num times of obs. } k \text{ in state } i}{\text{exp. num of trans. from } i} = \frac{\sum_{t=1}^{T-1} \gamma_t(i) \mathbb{1}_{O_t=k}}{\sum_{t=1}^{T-1} \gamma_t(i)}$$

Initial probabilities

$$\bar{\pi} = \text{exp. frequency in state } i \text{ at time } (t = 1) = \gamma_1(i)$$

Tips, Tricks & Reminders

References

- **Rabiner 1989 [Sec. I-III, Sec. V]:** My favorite HMM tutorial! <https://ieeexplore.ieee.org/document/18626>
- **Stamp tutorial:** (on Canvas) useful for implementation

Implementation (see Stamp tutorial for details)

- Use natural logarithms and summations instead of products
- Matrix initialization important for convergence of the Baum-Welch algorithm

Presentations

- Sign up on Canvas – don't forget to include Kattis submission ID!
- **BRING WRITTEN SOLUTIONS TO QUESTIONS**